

Low-Level Design (LLD) – Complete Guide

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

 **Study window:** 13 Dec 2026 – 3 Jan 2027 (Days 119-140)

 **Your slots (IST):** Mon-Fri **7:30-8:30pm** & **9:00-10:00pm** · Sat-Sun **9:00am-12:00pm, 3:00-5:30pm, 6:00-8:30pm**

 **Calendar reminders:** harsh.gupta@getreelax.com – import `calendar/mern-mastery-study.ics`

SOLID principles, design patterns, and full TypeScript implementations for classic interview problems.

Table of Contents

1. LLD Interview Process
2. SOLID Principles
3. Design Patterns
4. LRU Cache
5. Parking Lot System
6. Rate Limiter
7. Splitwise (Expense Sharing)
8. BookMyShow (Ticket Booking)
9. Interview Q&A
10. Practice Resources

1. LLD Interview Process

Timeline (45-60 Minutes)

0-5 min Clarify requirements, ask questions
5-10 min Identify entities, relationships, enums
10-15 min Draw class diagram (text/whiteboard)
15-40 min Code core classes and main flows
40-50 min Handle edge cases, extensibility
50-60 min Discuss trade-offs, testing, scale

Questions to Ask

- How many users / concurrent requests?
- Single instance or distributed?
- Persistence needed or in-memory OK?
- What are the core use cases vs nice-to-have?
- Error handling expectations?
- Thread safety required?

Evaluation Criteria

Area	What Interviewers Look For
Requirements	Clarifying questions, scope control
Design	Clean abstractions, appropriate patterns
Code	Working logic, readable TypeScript/Java
Extensibility	Open for new types without rewriting
Communication	Explaining trade-offs clearly

2. SOLID Principles

S – Single Responsibility Principle

A class should have only one reason to change.

```
// ❌ Bad: User class handles auth AND persistence AND email
class User {
  login() { /* ... */ }
  saveToDB() { /* ... */ }
  sendWelcomeEmail() { /* ... */ }
}

// ✅ Good: Separate concerns
class AuthService { login(user: User): Token { /* ... */ } }
class UserRepository { save(user: User): void { /* ... */ } }
class EmailService { sendWelcome(user: User): void { /* ... */ } }
```

O – Open/Closed Principle

Open for extension, closed for modification.

```
interface PaymentProcessor {
  process(amount: number): boolean;
}

class CreditCardProcessor implements PaymentProcessor {
  process(amount: number) { /* stripe */ return true; }
}

class UPIProcessor implements PaymentProcessor {
  process(amount: number) { /* razorpay */ return true; }
}

class PaymentService {
  constructor(private processor: PaymentProcessor) {}
  checkout(amount: number) { return this.processor.process(amount); }
}

// Add new payment method without changing PaymentService
```

L – Liskov Substitution Principle

Subtypes must be substitutable for base types.

```
abstract class Bird {
  abstract move(): void;
}

class Sparrow extends Bird {
  move() { console.log("Flying"); }
}

class Penguin extends Bird {
  move() { console.log("Swimming"); } } // ✅ Both can "move"
}

// ❌ Don't put fly() on Bird if Penguin can't fly
```

I – Interface Segregation Principle

Clients shouldn't depend on interfaces they don't use.

```
interface Readable { read(): string; }
interface Writable { write(data: string): void; }

class FileReader implements Readable {
  read() { return "content"; }
}

class FileWriter implements Writable {
  write(data: string) { /* ... */ }
}

// Not one fat IFile interface with read+write+delete+compress
```

D – Dependency Inversion Principle

Depend on abstractions, not concretions.

```
interface Cache {
  get(key: string): string | null;
  set(key: string, value: string): void;
}

class RedisCache implements Cache { /* ... */ }
class InMemoryCache implements Cache { /* ... */ }

class UserService {
  constructor(private cache: Cache) {} // injected abstraction
}
```

3. Design Patterns

Creational

Pattern	Use Case	Example
Singleton	One instance (DB pool, config)	Logger, RateLimiter
Factory	Create objects without specifying class	VehicleFactory
Builder	Complex object step-by-step	SQL query builder

```
// Singleton
class DatabaseConnection {
  private static instance: DatabaseConnection;
  private constructor() {}
  static getInstance(): DatabaseConnection {
    if (!DatabaseConnection.instance) {
      DatabaseConnection.instance = new DatabaseConnection();
    }
    return DatabaseConnection.instance;
  }
}
```

Structural

Pattern	Use Case	Example
Adapter	Make incompatible interfaces work	Payment gateway wrapper
Decorator	Add behavior dynamically	CachedRepository wraps DB
Facade	Simplified interface to subsystem	BookingFacade

Behavioral

Pattern	Use Case	Example
Strategy	Swap algorithms at runtime	PricingStrategy
Observer	Notify subscribers on event	OrderStatusNotifier
Command	Encapsulate request as object	Undo/redo operations

```
// Strategy Pattern
interface PricingStrategy {
  calculate(basePrice: number): number;
}

class RegularPricing implements PricingStrategy {
  calculate(base: number) { return base; }
}

class PremiumPricing implements PricingStrategy {
  calculate(base: number) { return base * 0.8; }
}

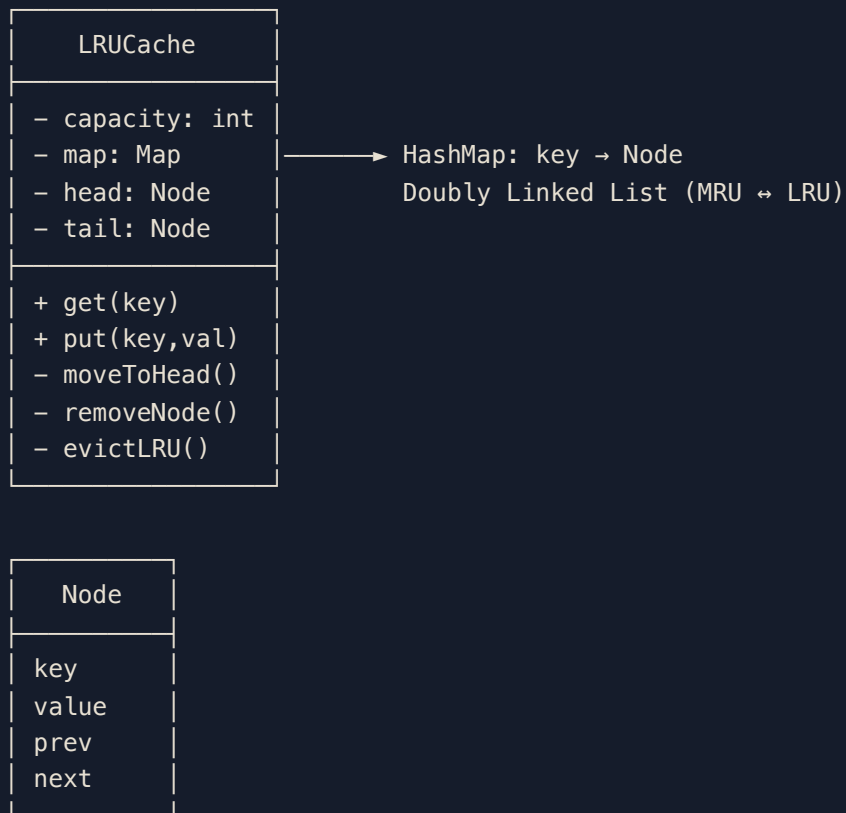
class PriceCalculator {
  constructor(private strategy: PricingStrategy) {}
  getPrice(base: number) { return this.strategy.calculate(base); }
}
```

4. LRU Cache

Requirements

- Fixed capacity
- `get(key)` — return value, mark as recently used; -1 if missing
- `put(key, value)` — insert/update; evict LRU if at capacity
- Both operations $O(1)$

Class Diagram (Text)



Full TypeScript Implementation

```
class DLLNode {
  key: number;
  value: number;
  prev: DLLNode | null = null;
  next: DLLNode | null = null;

  constructor(key: number, value: number) {
    this.key = key;
    this.value = value;
  }
}

class LRUCache {
  private capacity: number;
  private map = new Map<number, DLLNode>();
  private head: DLLNode;
  private tail: DLLNode;

  constructor(capacity: number) {
    this.capacity = capacity;
    this.head = new DLLNode(0, 0);
    this.tail = new DLLNode(0, 0);
    this.head.next = this.tail;
    this.tail.prev = this.head;
  }

  get(key: number): number {
    if (!this.map.has(key)) return -1;
    const node = this.map.get(key)!;
    this.moveToHead(node);
    return node.value;
  }

  put(key: number, value: number): void {
    if (this.map.has(key)) {
      const node = this.map.get(key)!;
      node.value = value;
      this.moveToHead(node);
      return;
    }

    const node = new DLLNode(key, value);
    this.map.set(key, node);
    this.addToHead(node);

    if (this.map.size > this.capacity) {
      const lru = this.removeTail();
      this.map.delete(lru.key);
    }
  }
}
```

```
    }  
  }  
  
  private addToHead(node: DLLNode): void {  
    node.prev = this.head;  
    node.next = this.head.next;  
    this.head.next!.prev = node;  
    this.head.next = node;  
  }  
  
  private removeNode(node: DLLNode): void {  
    node.prev!.next = node.next;  
    node.next!.prev = node.prev;  
  }  
  
  private moveToHead(node: DLLNode): void {  
    this.removeNode(node);  
    this.addToHead(node);  
  }  
  
  private removeTail(): DLLNode {  
    const node = this.tail.prev!;  
    this.removeNode(node);  
    return node;  
  }  
}
```

Complexity: get $O(1)$, put $O(1)$ | Space $O(\text{capacity})$

5. Parking Lot System

Requirements

- Multiple floors, multiple spot types (compact, large, handicapped)
- Park vehicle (car, motorcycle, truck)
- Unpark vehicle, calculate fee
- Display availability
- Spot assignment: nearest available compatible spot

Class Diagram (Text)



TypeScript Implementation (Core)

```
enum SpotType { COMPACT, LARGE, HANDICAPPED }
enum VehicleType { MOTORCYCLE, CAR, TRUCK }

abstract class Vehicle {
  constructor(public plate: string, public type: VehicleType) {}
}

class Car extends Vehicle {
  constructor(plate: string) { super(plate, VehicleType.CAR); }
}

class ParkingSpot {
  vehicle: Vehicle | null = null;
  constructor(public id: string, public type: SpotType) {}

  canFit(vehicle: Vehicle): boolean {
    if (this.vehicle) return false;
    if (vehicle.type === VehicleType.MOTORCYCLE) return true;
    if (vehicle.type === VehicleType.CAR)
      return this.type === SpotType.COMPACT || this.type === SpotType.LARGE;
    if (vehicle.type === VehicleType.TRUCK) return this.type === SpotType.LARGE;
    return false;
  }

  park(vehicle: Vehicle): boolean {
    if (!this.canFit(vehicle)) return false;
    this.vehicle = vehicle;
    return true;
  }

  unpark(): Vehicle | null {
    const v = this.vehicle;
    this.vehicle = null;
    return v;
  }
}

class Floor {
  constructor(public level: number, public spots: ParkingSpot[]) {}

  findSpot(vehicle: Vehicle): ParkingSpot | null {
    return this.spots.find(s => s.canFit(vehicle)) ?? null;
  }
}

class Ticket {
  constructor(
    public id: string,
```

```
    public vehicle: Vehicle,
    public spot: ParkingSpot,
    public entryTime: Date = new Date()
  ) {}
}

class ParkingLot {
  private static instance: ParkingLot;
  private tickets = new Map<string, Ticket>();

  private constructor(public floors: Floor[]) {}

  static getInstance(floors?: Floor[]): ParkingLot {
    if (!ParkingLot.instance) {
      if (!floors) throw new Error("Initialize with floors first");
      ParkingLot.instance = new ParkingLot(floors);
    }
    return ParkingLot.instance;
  }

  park(vehicle: Vehicle): Ticket | null {
    for (const floor of this.floors) {
      const spot = floor.findSpot(vehicle);
      if (spot?.park(vehicle)) {
        const ticket = new Ticket(crypto.randomUUID(), vehicle, spot);
        this.tickets.set(ticket.id, ticket);
        return ticket;
      }
    }
    return null;
  }

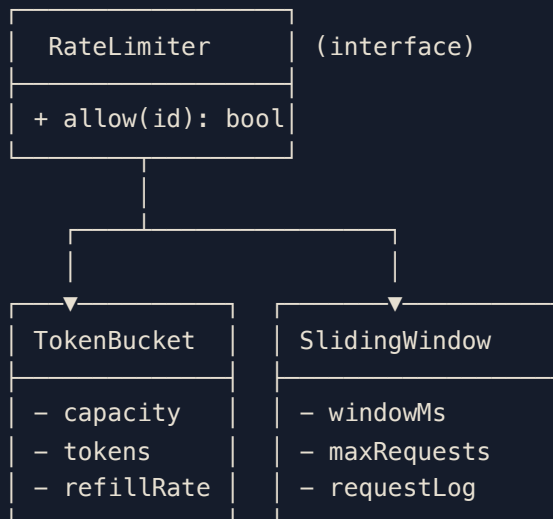
  unpark(ticketId: string): number {
    const ticket = this.tickets.get(ticketId);
    if (!ticket) throw new Error("Invalid ticket");
    ticket.spot.unpark();
    this.tickets.delete(ticketId);
    const hours = (Date.now() - ticket.entryTime.getTime()) / 3600000;
    return Math.ceil(hours) * 50; // ₹50 per hour
  }
}
```

6. Rate Limiter

Requirements

- Limit requests per user/IP per time window
- Algorithms: Fixed Window, Sliding Window, Token Bucket
- `allowRequest(clientId): boolean`
- Thread-safe for concurrent access

Class Diagram (Text)



Token Bucket Implementation

```
class TokenBucketLimiter {
  private buckets = new Map<string, { tokens: number; lastRefill: number }>();

  constructor(
    private capacity: number,
    private refillRate: number // tokens per second
  ) {}

  allowRequest(clientId: string): boolean {
    const now = Date.now();
    let bucket = this.buckets.get(clientId);

    if (!bucket) {
      bucket = { tokens: this.capacity, lastRefill: now };
      this.buckets.set(clientId, bucket);
    }

    const elapsed = (now - bucket.lastRefill) / 1000;
    bucket.tokens = Math.min(this.capacity, bucket.tokens + elapsed * this.refillRate);
    bucket.lastRefill = now;

    if (bucket.tokens >= 1) {
      bucket.tokens -= 1;
      return true;
    }
    return false;
  }
}

// Middleware usage
function rateLimitMiddleware(limiter: TokenBucketLimiter) {
  return (req: Request, res: Response, next: NextFunction) => {
    const clientId = req.ip ?? "unknown";
    if (!limiter.allowRequest(clientId)) {
      return res.status(429).json({ error: "Too many requests" });
    }
    next();
  };
}
```

Sliding Window Log

```
class SlidingWindowLimiter {
  private logs = new Map<string, number[]>();

  constructor(
    private windowMs: number,
    private maxRequests: number
  ) {}

  allowRequest(clientId: string): boolean {
    const now = Date.now();
    const windowStart = now - this.windowMs;
    let timestamps = this.logs.get(clientId) ?? [];
    timestamps = timestamps.filter(t => t > windowStart);

    if (timestamps.length >= this.maxRequests) {
      this.logs.set(clientId, timestamps);
      return false;
    }

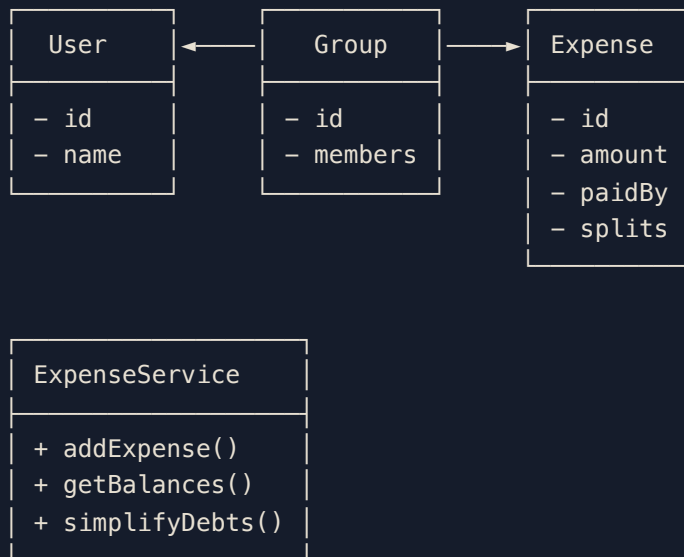
    timestamps.push(now);
    this.logs.set(clientId, timestamps);
    return true;
  }
}
```

7. Splitwise (Expense Sharing)

Requirements

- Users, groups
- Add expense: equal split, exact amounts, percentage
- Show balances between users
- Simplify debts (minimize transactions)

Class Diagram (Text)



TypeScript Implementation

```
enum SplitType { EQUAL, EXACT, PERCENTAGE }

interface Split {
  userId: string;
  amount: number;
}

class Expense {
  constructor(
    public id: string,
    public groupId: string,
    public paidBy: string,
    public totalAmount: number,
    public splits: Split[],
    public description: string
  ) {}
}

class ExpenseService {
  private expenses: Expense[] = [];
  // balanceMap: debtor → creditor → amount owed
  private balances = new Map<string, Map<string, number>>();

  addExpense(
    groupId: string,
    paidBy: string,
    totalAmount: number,
    splitType: SplitType,
    participants: string[],
    customSplits?: Split[]
  ): Expense {
    let splits: Split[];

    switch (splitType) {
      case SplitType.EQUAL:
        const share = totalAmount / participants.length;
        splits = participants.map(id => ({ userId: id, amount: share }));
        break;
      case SplitType.EXACT:
        splits = customSplits!;
        break;
      case SplitType.PERCENTAGE:
        splits = customSplits!.map(s => ({
          userId: s.userId,
          amount: (s.amount / 100) * totalAmount,
        }));
        break;
    }
  }
}
```

```

    const expense = new Expense(
      crypto.randomUUID(), groupId, paidBy, totalAmount, splits, ""
    );
    this.expenses.push(expense);
    this.updateBalances(paidBy, splits);
    return expense;
  }

  private updateBalances(paidBy: string, splits: Split[]): void {
    for (const split of splits) {
      if (split.userId === paidBy) continue;
      this.addDebt(split.userId, paidBy, split.amount);
    }
  }

  private addDebt(from: string, to: string, amount: number): void {
    if (!this.balances.has(from)) this.balances.set(from, new Map());
    const debts = this.balances.get(from)!;
    debts.set(to, (debts.get(to) ?? 0) + amount);
  }

  simplifyDebts(): { from: string; to: string; amount: number }[] {
    const net = new Map<string, number>();

    for (const [debtor, creditors] of this.balances) {
      for (const [creditor, amount] of creditors) {
        net.set(debtor, (net.get(debtor) ?? 0) - amount);
        net.set(creditor, (net.get(creditor) ?? 0) + amount);
      }
    }

    const debtors = [...net.entries()].filter(([, v]) => v < 0).sort((a, b) => a[1] < b[1]);
    const creditors = [...net.entries()].filter(([, v]) => v > 0).sort((a, b) => b[1] < a[1]);

    const result: { from: string; to: string; amount: number }[] = [];
    let i = 0, j = 0;

    while (i < debtors.length && j < creditors.length) {
      const amount = Math.min(-debtors[i][1], creditors[j][1]);
      result.push({ from: debtors[i][0], to: creditors[j][0], amount });
      debtors[i][1] += amount;
      creditors[j][1] -= amount;
      if (debtors[i][1] === 0) i++;
      if (creditors[j][1] === 0) j++;
    }
    return result;
  }
}

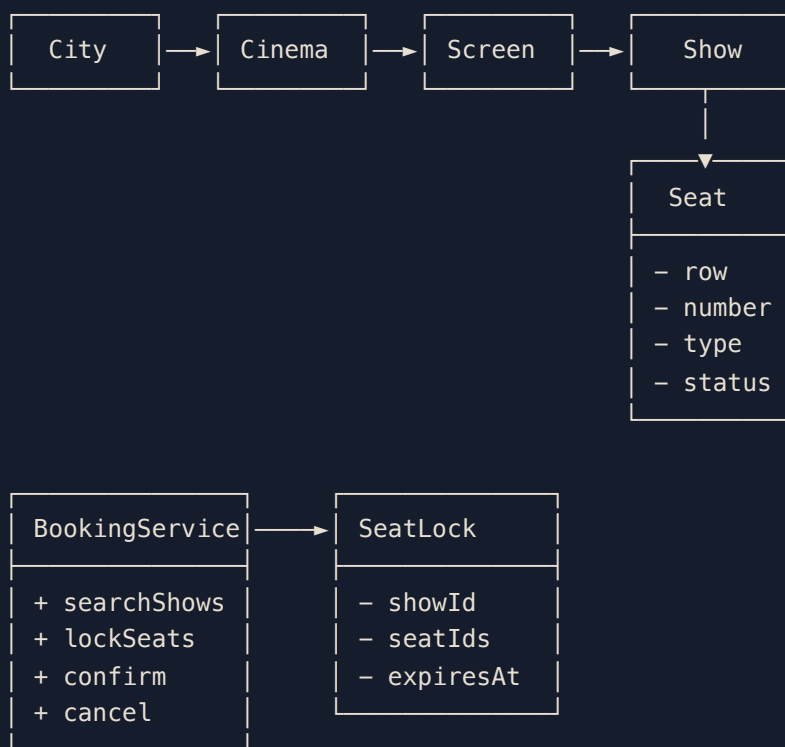
```

8. BookMyShow (Ticket Booking)

Requirements

- Cities, cinemas, screens, shows
- Seat layout with types (silver, gold, platinum)
- Concurrent booking — no double booking
- Lock seats temporarily (5 min hold)
- Payment confirmation

Class Diagram (Text)



```
enum SeatStatus { AVAILABLE, LOCKED, BOOKED }
```

TypeScript Implementation (Core)

```
enum SeatStatus { AVAILABLE, LOCKED, BOOKED }
enum SeatType { SILVER, GOLD, PLATINUM }

class Seat {
  status: SeatStatus = SeatStatus.AVAILABLE;
  constructor(
    public id: string,
    public row: number,
    public number: number,
    public type: SeatType
  ) {}
}

class Show {
  seats: Seat[];
  constructor(
    public id: string,
    public movieId: string,
    public startTime: Date,
    seatLayout: Seat[]
  ) {
    this.seats = seatLayout;
  }

  getAvailableSeats(): Seat[] {
    return this.seats.filter(s => s.status === SeatStatus.AVAILABLE);
  }
}

class SeatLock {
  constructor(
    public lockId: string,
    public showId: string,
    public seatIds: string[],
    public userId: string,
    public expiresAt: Date
  ) {}

  isExpired(): boolean {
    return Date.now() > this.expiresAt.getTime();
  }
}

class BookingService {
  private locks = new Map<string, SeatLock>();
  private shows = new Map<string, Show>();

  lockSeats(showId: string, seatIds: string[], userId: string): SeatLock | null {
```

```
const show = this.shows.get(showId);
if (!show) return null;

// Atomic check – in production use DB transaction + row lock
for (const seatId of seatIds) {
  const seat = show.seats.find(s => s.id === seatId);
  if (!seat || seat.status !== SeatStatus.AVAILABLE) return null;
}

for (const seatId of seatIds) {
  const seat = show.seats.find(s => s.id === seatId)!;
  seat.status = SeatStatus.LOCKED;
}

const lock = new SeatLock(
  crypto.randomUUID(),
  showId,
  seatIds,
  userId,
  new Date(Date.now() + 5 * 60 * 1000)
);
this.locks.set(lock.lockId, lock);
return lock;
}

confirmBooking(lockId: string, paymentId: string): boolean {
  const lock = this.locks.get(lockId);
  if (!lock || lock.isExpired()) {
    this.releaseLock(lockId);
    return false;
  }

  const show = this.shows.get(lock.showId)!;
  for (const seatId of lock.seatIds) {
    const seat = show.seats.find(s => s.id === seatId)!;
    seat.status = SeatStatus.BOOKED;
  }
  this.locks.delete(lockId);
  return true;
}

releaseLock(lockId: string): void {
  const lock = this.locks.get(lockId);
  if (!lock) return;
  const show = this.shows.get(lock.showId);
  if (show) {
    for (const seatId of lock.seatIds) {
      const seat = show.seats.find(s => s.id === seatId);
      if (seat?.status === SeatStatus.LOCKED) seat.status = SeatStatus.AVAILABLE;
    }
  }
  this.locks.delete(lockId);
}
```

```
}  
}
```

Concurrency note: Production systems use Redis distributed locks or DB `SELECT FOR UPDATE`.

9. Interview Q&A

Q: When to use Singleton? A: When exactly one instance is needed (config, connection pool). Be cautious in distributed systems — use external singleton (Redis, DB).

Q: Strategy vs Factory pattern? A: Factory creates objects; Strategy selects algorithm/behavior at runtime. Often used together.

Q: How to handle concurrency in seat booking? A: Optimistic locking (version field), pessimistic locking (DB row lock), distributed locks (Redis Redlock), or queue-based serialization.

Q: LRU Cache — why doubly linked list + hash map? A: Map gives $O(1)$ lookup; DLL gives $O(1)$ insert/remove at any position for move-to-front and evict-from-back.

Q: How to extend Parking Lot for electric charging spots? A: Add `ElectricSpot` extends `ParkingSpot` or new `SpotType` enum value; override `canFit()` if needed. Open/Closed principle.

Q: Token Bucket vs Sliding Window rate limiter? A: Token bucket allows bursts (smooth refill); sliding window is precise per window but uses more memory (stores timestamps).

10. Practice Resources

Resource	URL	Focus
InterviewReady LLD	Various playlists	Parking Lot, Elevator
GitHub LLD Solutions	Search "machine coding"	TypeScript/Java repos
Educative Grokking LLD	educative.io	Structured course
LeetCode Discuss	LLD tagged posts	Community solutions

Practice Problems

1. LRU Cache (implement from scratch in 20 min)
2. Parking Lot with 3 vehicle types

3. Rate Limiter — Token Bucket + middleware
 4. Splitwise with debt simplification
 5. BookMyShow with seat locking
 6. Elevator System
 7. Vending Machine (state pattern)
 8. Logger (Singleton + strategy for appenders)
 9. Tic-Tac-Toe / Chess (OOP design)
 10. Library Management System
-

Quick Reference

SOLID → Single responsibility, Open/closed, Liskov, Interface segregation, Dependency
Patterns → Singleton, Factory, Strategy, Observer most common in LLD
Process → Clarify → Entities → Diagram → Code → Edge cases → Trade-offs
Always → Ask questions, start simple, extensible enums/interfaces

Low Level Design Interview – 100 Questions

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

This bank covers object-oriented design, design patterns, SOLID principles, and classic Low Level Design (LLD) problems asked at product companies. Spell out abbreviations on first use.

1. What is Low Level Design (LLD)?

Low Level Design is the process of designing the internal structure of a software system at the class and object level. It focuses on how individual components interact, which classes to create, which design patterns to apply, and how to follow Object-Oriented Programming (OOP) principles. In product company interviews, LLD tests whether you can turn a vague requirement like 'design a parking lot' into clean, extensible code.

2. What are the four pillars of Object-Oriented Programming (OOP)?

The four pillars are: **Encapsulation** (hiding internal state behind methods), **Abstraction** (showing only essential details), **Inheritance** (reusing behavior from parent classes), and **Polymorphism** (same interface, different implementations). Together they help you model real-world entities and write maintainable code.

3. What is encapsulation and why does it matter in LLD interviews?

Encapsulation means bundling data and the methods that operate on that data inside a class, and restricting direct access to fields (usually via private/protected modifiers and public getters/setters). It prevents callers from putting objects into invalid states. In a parking lot design, a `Spot` class should not let external code set `isOccupied = true` without going through a `park(Vehicle)` method that validates availability.

4. What is abstraction in OOP?

Abstraction hides complex implementation details and exposes only what the caller needs. For example, a `PaymentProcessor` interface might expose `charge(amount)` while hiding whether the backend uses Stripe, Razorpay, or a mock gateway. Abstraction reduces coupling and makes systems easier to test and extend.

5. Explain inheritance with a real LLD example.

Inheritance lets a child class reuse and extend a parent class. In a parking lot, you might have a base `Vehicle` class with `licensePlate` and `size`, and subclasses `Car`, `Motorcycle`, and `Truck` that override `getSpotType()`. This avoids duplicating common logic while allowing type-specific behavior.

6. What is polymorphism and how is it used in design?

Polymorphism means 'many forms.' A parent type reference can point to any child object, and the correct method runs at runtime (dynamic dispatch). Example: `List<Vehicle> vehicles` where each vehicle's `getSpotType()` returns different values. Polymorphism enables open/closed design—you add new vehicle types without changing existing parking logic.

7. What is composition over inheritance?

Composition means building behavior by combining objects ('has-a') rather than extending classes ('is-a'). A `ParkingLot` has a `PricingStrategy` and a `SpotAllocator`, instead of inheriting from them. Composition is preferred because deep inheritance hierarchies become rigid and fragile; composition allows swapping behavior at runtime.

8. What does SOLID stand for?

SOLID is five design principles: **S**ingle Responsibility, **O**pen/Closed, **L**iskov Substitution, **I**nterface Segregation, and **D**ependency Inversion. They guide you toward modular, testable, and extensible designs. Interviewers expect you to mention relevant SOLID principles while designing classes.

9. Explain the Single Responsibility Principle (SRP).

SRP states that a class should have only one reason to change. Example: `InvoiceGenerator` creates PDF invoices; `PaymentService` processes payments. Mixing both in one class means a change to PDF formatting also risks breaking payment logic. In LLD, split responsibilities into focused classes.

10. Explain the Open/Closed Principle (OCP).

OCP says software entities should be open for extension but closed for modification. Add new behavior via new classes or strategies, not by editing existing code. Example: add `PremiumPricingStrategy` without changing `ParkingLot`—inject the strategy through an interface.

11. Explain the Liskov Substitution Principle (LSP).

LSP says subtypes must be substitutable for their base types without breaking correctness. If `Square` extends `Rectangle` but changing width does not update height consistently, callers expecting rectangle behavior will break. In LLD, ensure subclasses honor the contract of the parent interface.

12. Explain the Interface Segregation Principle (ISP).

ISP means clients should not depend on interfaces they do not use. Instead of one fat `Worker` interface with `code()`, `manage()`, and `clean()`, split into `Developer`, `Manager`, and `Janitor` interfaces. This keeps implementations lean and avoids empty stub methods.

13. Explain the Dependency Inversion Principle (DIP).

DIP says high-level modules should not depend on low-level modules; both should depend on abstractions. A `BookingService` should depend on a `PaymentGateway` interface, not on `StripePayment` directly. This enables swapping implementations and simplifies unit testing with mocks.

14. What is a design pattern?

A design pattern is a reusable solution to a commonly occurring problem in software design. Patterns are not copy-paste code—they are templates for structure and communication between

objects. Common categories: Creational, Structural, and Behavioral. Interviewers want you to recognize when a pattern fits, not force one everywhere.

15. Explain the Singleton pattern. When should you avoid it?

Singleton ensures only one instance of a class exists (e.g., a configuration manager). Implementation uses a private constructor and a static `getInstance()` method. Avoid Singleton in interviews unless truly needed—it hides dependencies, complicates testing, and can cause thread-safety issues. Prefer dependency injection instead.

16. Explain the Factory Method pattern.

Factory Method defines an interface for creating objects but lets subclasses decide which class to instantiate. Example: `VehicleFactory.create(type)` returns `Car`, `Bike`, or `Truck`. Callers depend on the factory interface, not concrete constructors. Useful when object creation logic is complex or varies by input.

17. Explain the Abstract Factory pattern.

Abstract Factory provides an interface for creating families of related objects without specifying concrete classes. Example: `UIFactory` creates matching `Button` and `Dialog` for Windows vs Mac. Different from Factory Method, which creates one product; Abstract Factory creates a coordinated set.

18. Explain the Builder pattern.

Builder separates construction of a complex object from its representation. Example: `Pizza.Builder().crust('thin').topping('pepperoni').build()`. Useful when an object has many optional parameters—cleaner than telescoping constructors. Common in machine coding for entities with many fields.

19. Explain the Strategy pattern.

Strategy defines a family of algorithms, encapsulates each one, and makes them interchangeable. Example: `PricingStrategy` with `HourlyPricing`, `FlatRatePricing`, and `WeekendPricing`. The context (`ParkingLot`) delegates pricing to the injected strategy. This is one of the most useful patterns in LLD.

20. Explain the Observer pattern.

Observer defines a one-to-many dependency: when one object changes state, all dependents are notified. Example: `OrderStatus` notifies `EmailNotifier`, `SMSNotifier`, and `InventoryService` when status changes. Foundation for event-driven UI and pub/sub systems.

21. Explain the Decorator pattern.

Decorator attaches additional responsibilities to an object dynamically without subclassing. Example: `BufferedInputStream` wraps `FileInputStream` to add buffering. In application code, wrap a base service with `LoggingService` or `CachingService` decorators.

22. Explain the Adapter pattern.

Adapter converts the interface of a class into another interface clients expect. Example: your app expects `PaymentGateway.charge()`, but a third-party library exposes `processPayment()`. An adapter wraps the library and exposes your interface. Common when integrating legacy or external APIs.

23. Explain the Facade pattern.

Facade provides a simplified interface to a complex subsystem. Example: `BookingFacade.bookTicket()` internally calls seat selection, payment, notification, and inventory modules. Clients interact with one entry point instead of many services.

24. Explain the Command pattern.

Command encapsulates a request as an object, enabling undo, queuing, and logging. Example: text editor commands `CopyCommand`, `PasteCommand`, `UndoCommand` each implement `execute()` and `undo()`. Useful in machine coding when actions must be reversible or scheduled.

25. What is an Enum and when is it better than constants?

An Enum (enumeration) is a type-safe set of named constants. `enum SpotType { COMPACT, LARGE, HANDICAPPED }` is safer than string constants because the compiler catches typos and you can attach

methods to enum values. Use enums for fixed, known sets like status, vehicle type, or payment mode.

26. How do you design a Parking Lot system in LLD?

Identify entities: `ParkingLot`, `Level`, `ParkingSpot`, `Vehicle`, `Ticket`, `Payment`. Define relationships: a lot has many levels; each level has spots; spots hold one vehicle. Core operations: `park(vehicle)`, `unpark(ticketId)`, `getAvailableSpots(type)`. Apply Strategy for pricing, Factory for vehicle/spot types, and Singleton sparingly for lot configuration. Discuss concurrency if multiple entry gates exist.

27. How do you handle different vehicle sizes in a parking lot design?

Model `VehicleSize` enum (SMALL, MEDIUM, LARGE). Spots are typed—compact spots fit motorcycles and small cars; large spots fit trucks. Allocation logic: try exact match first, then next larger spot if policy allows (e.g., a car may use a large spot but not vice versa). Keep allocation in a dedicated `SpotAllocator` class (SRP).

28. How would you extend a parking lot to support electric vehicle (EV) charging spots?

Add `EVSpot` extending or composing with `ParkingSpot`, adding `chargingPort` and `isCharging`. Extend allocator to filter EV-capable spots for electric vehicles. Add `ChargingBillingStrategy` for time + energy fees. OCP: new spot type and strategy without rewriting core parking flow.

29. What classes and interfaces would you define for a Rate Limiter?

Core: `RateLimiter` interface with `allowRequest(userId)` returning boolean. Implementations: `TokenBucketLimiter`, `FixedWindowLimiter`, `SlidingWindowLimiter`. Supporting: `Clock` (for testability), `Storage` (in-memory map or Redis-backed). Optional: `RateLimitConfig` with limits per tier. DIP: middleware depends on `RateLimiter`, not a concrete algorithm.

30. Explain the Token Bucket algorithm for rate limiting.

Token Bucket maintains a bucket with a maximum capacity and a refill rate (tokens per second). Each request consumes one token; if the bucket is empty, the request is rejected. Burst traffic is allowed up to bucket capacity, then rate smooths to refill rate. Implementation: track `tokens` and `lastRefillTime`; on each request, refill based on elapsed time, then deduct.

31. Explain Fixed Window vs Sliding Window rate limiting.

Fixed Window divides time into buckets (e.g., 100 requests per minute starting at :00). Simple but allows bursts at window boundaries (200 requests in 2 seconds across two windows). Sliding Window tracks requests in a rolling time frame (e.g., last 60 seconds), smoother but needs more storage (timestamps or sub-window counters). Sliding Window Log stores each request timestamp; Sliding Window Counter hybridizes.

32. How do you design an LRU (Least Recently Used) Cache?

LRU Cache supports `get(key)` and `put(key, value)` in $O(1)$ average time with a capacity limit. When full, evict the least recently used entry. Classic structure: `HashMap` for $O(1)$ lookup + Doubly Linked List for $O(1)$ move-to-front and eviction. On `get` or `put`, move node to head; evict tail when size exceeds capacity.

33. Walk through LRU Cache operations step by step.

`get(key)` : if missing, return -1; if present, move node to head (most recent), return value.
`put(key, value)` : if key exists, update value and move to head; if new and at capacity, remove tail node and its map entry, then insert new node at head. Use dummy head/tail sentinel nodes to simplify edge cases. Time: $O(1)$; Space: $O(\text{capacity})$.

34. How would you make LRU Cache thread-safe?

Wrap critical sections with `synchronized` blocks or `ReentrantReadWriteLock` (read lock for `get`, write lock for `put`). Alternatively use `ConcurrentHashMap` plus a lock on list mutations. For high concurrency, consider segmented caches or lock-free structures—but for interviews, explain trade-offs clearly.

35. What is the difference between an interface and an abstract class?

An interface defines a contract (method signatures) with no state; a class can implement multiple interfaces. An abstract class can have both abstract and concrete methods plus shared fields; a class inherits one abstract class. Use interfaces for capability contracts (`Payable` , `Serializable`); abstract classes when sharing base implementation among closely related types.

36. What is dependency injection (DI)?

Dependency Injection means providing an object's dependencies from the outside rather than creating them internally. Constructor injection is preferred: `ParkingLot(pricingStrategy, allocator)` . Benefits: testability (inject mocks), flexibility (swap implementations), and explicit dependencies. Frameworks like Spring automate DI via annotations.

37. What is the difference between aggregation and composition?

Both are 'has-a' relationships. **Aggregation** is a weak relationship—the child can exist independently (Department has Employees; employees can transfer). **Composition** is strong—the child cannot exist without the parent (House has Rooms; destroying the house destroys rooms). Composition implies lifecycle ownership.

38. How do you identify entities vs value objects in LLD?

Entities have identity that persists over time (User, Order, Ticket)—two orders with same data are still different orders. Value objects are defined by their attributes (Money, Address, DateRange)—two `$10 USD` values are interchangeable. Entities get unique IDs; value objects should be immutable.

39. What is immutability and why is it useful?

An immutable object's state cannot change after creation. All fields are final, no setters, and nested objects are defensive copies. Benefits: thread safety without locks, safer sharing, easier reasoning. Use immutable value objects like `Money` , `Coordinates` , or `TicketSnapshot` in LLD designs.

40. How do you handle errors in LLD design?

Define a clear exception hierarchy: `ParkingFullException`, `InvalidTicketException`, `PaymentFailedException`. Do not use generic exceptions for business cases. Return typed results where appropriate (`Optional<Spot>`, `Result<T, E>`). Separate validation (input checks) from domain rules (no spot available). Document which methods throw what.

41. What is the Template Method pattern?

Template Method defines the skeleton of an algorithm in a base class, letting subclasses override specific steps. Example: `DataExporter.export()` calls `readData()`, `transform()`, `write()` — subclasses override steps but not the flow. Useful when multiple processes share the same sequence with varying details.

42. What is the State pattern?

State lets an object alter its behavior when its internal state changes, as if the object changed class. Example: `Order` with states `Placed`, `Shipped`, `Delivered`, `Cancelled` —each state implements `cancel()` differently. Avoids large if/else chains on status enums.

43. How would you design a Library Management System in LLD?

Entities: `Book`, `BookItem` (physical copy with barcode), `Member`, `Lending`, `Reservation`, `Fine`. Services: `CatalogService`, `LendingService`, `SearchService`. Operations: search, checkout, return, reserve, calculate fine. Apply Observer for due-date reminders; Strategy for fine calculation (daily rate vs flat fee).

44. How would you design an Elevator system in LLD?

Entities: `Elevator`, `Floor`, `Request` (external up/down, internal floor button), `ElevatorController`. Algorithm: SCAN or LOOK for efficient movement. State pattern for elevator direction (UP, DOWN, IDLE). Controller dispatches requests to nearest idle elevator. Discuss thread safety for concurrent button presses.

45. What is an anti-pattern you should avoid in LLD interviews?

God Object: one class that knows and does everything. Solution: apply SRP and split into services. Also avoid: premature Singleton, deep inheritance trees, and anemic domain models (data classes with all logic in separate service classes with no behavior in entities).

46. How do you communicate your LLD design in an interview?

Follow: (1) Clarify requirements and scope, (2) Identify nouns → entities, verbs → methods, (3) Draw class diagram with relationships, (4) Define key interfaces and one happy-path walkthrough, (5) Discuss extensibility and edge cases. Write clean pseudocode or real code for 2–3 core methods. Ask the interviewer which part to deep-dive.

47. What is the difference between LLD and High Level Design (HLD)?

LLD focuses on class-level structure, design patterns, and code organization for a single service or module. HLD focuses on system-wide architecture: services, databases, APIs, scaling, and data flow across components. LLD answers 'how does this class work?'; HLD answers 'how do these services talk to each other?'

48. How do you make an LLD design extensible for future features?

Use interfaces for volatile parts (pricing, notification, storage). Favor composition and strategy over conditionals. Keep domain models rich but services thin. Document extension points explicitly. Example: adding valet parking means new `ValetService` implementing `ParkingService`, not editing `ParkingLot` internals.

49. What testing approach supports good LLD?

Unit test each class in isolation with mocked dependencies. Test domain rules (cannot park in occupied spot), boundary cases (lot full), and strategy swapping. Dependency injection makes mocking trivial. Integration tests cover full flows (park → pay → unpark).

50. What are common follow-up questions after a parking lot LLD?

Concurrency at entry gates, handling payment failure mid-session, multi-floor allocation optimization, display board for available spots, handicap/reserved spots, and scaling to multiple parking lots with a central system. Be ready to add a `DisplayBoard` `Observer` or a `LotRegistry` for multi-site management.

Questions 51-100 (top product companies)

51. How do you show Open/Closed on parking lot?

New vehicle type = new class + spot rule, not a growing switch. Amazon / Flipkart.

52. Strategy vs Factory – one example each.

Strategy: fare calculation. Factory: create Vehicle from type string. Amazon.

53. Why is Singleton often an anti-pattern?

Hidden global, hard tests, hidden coupling. Prefer injection. Microsoft / Amazon.

54. How do you model a library management system?

Book, Copy, Member, Loan, Reservation. Copy has state. Adobe / Amazon.

55. Observer in a stock-ticker Low Level Design.

Ticker notifies subscribers. Careful of register/unregister leaks. Microsoft.

56. How do you design an ATM?

Card, session, states, dispenser, bank adapter interface. Amazon.

57. Decorator vs inheritance for pizza toppings.

Decorator adds cost/description without a class explosion. Microsoft.

58. How do you design a chess game (lite)?

Board, Piece.move(), turn, check. Do not build AI. Amazon / Uber.

59. Command pattern for a remote / undo.

Each action is an object with execute/undo. Editor undo stack. Adobe.

60. How do you design a hotel booking Low Level Design?

Room, Reservation, date range overlap check. Like intervals. Flipkart / Booking.

61. Adapter when wrapping a legacy payment API.

Your `Payable` interface; adapter maps old SDK. Razorpay / Adobe.

62. How do you test a rate limiter class?

Fake clock, burst, then reject, then refill. Uber / Amazon.

63. What is a god class and how do you split it?

ParkingLot doing pricing, SMS, and persistence — split. Flipkart bar.

64. How do you design a cache with Time To Live?

Expiry on node, lazy delete on get, optional sweep. Amazon.

65. Null object pattern – when?

Missing discount = NoDiscount with zero amount. Avoid null checks. Microsoft.

66. How do you design Snake and Ladder extensibly?

Board cells with optional teleport; Dice interface. Amazon / Flipkart.

67. Template method for data import.

Abstract parse/validate/save; CSV vs JSON subclasses. Adobe.

68. How do you handle concurrency in LRU?

Mutex around get/put or concurrent map + list policy. Amazon follow-up.

69. Builder for a complex search filter.

Fluent API, then immutable Filter. Atlassian / Elastic-like.

70. How do you design a notification dispatcher Low Level Design?

Channel interface, Email/Push impls, retry policy. Amazon / Flipkart.

71. State vs strategy – do not mix them up.

State: object changes behavior as it changes state. Strategy: caller injects algorithm. Amazon.

72. How do you design a scoring/leaderboard?

User score, heap or sorted set, getTop(K). Uber / Dream11.

73. Facade for a checkout flow.

One Checkout.place() hiding inventory, pay, notify. Flipkart / Amazon.

74. How do you version a public Low Level Design API?

Stable interfaces; new methods on new interfaces. Microsoft.

75. Composite for a file system.

File and Folder share Node; size recurses. Google / Adobe.

76. How do you design a meeting scheduler?

Intervals, conflict check, rooms heap. Amazon / Uber.

77. Dependency injection in plain TypeScript.

Pass interfaces into constructors; no `new Postgres()` inside. Adobe / Amazon.

78. How do you design a logger with levels?

Logger interface, console/file sinks, filter by level. Microsoft.

79. Memento for undo in an editor.

Store snapshots or commands. Adobe.

80. How do you design a vending machine inventory?

SKU, count, restock; state machine for money. Amazon.

81. Flyweight when?

Share glyph/style objects for a million characters. Rare Adobe.

82. How do you design a ride-sharing matching class (not HLD)?

Driver, Rider, Trip, Matcher interface. Uber.

83. Chain of responsibility for auth/validation.

Each handler can pass or stop. Express-like. Adobe.

84. How do you keep machine coding extensible in the last 10 minutes?

Say the next type and the one interface you would add. Flipkart.

85. What is Law of Demeter in your parking lot?

Do not reach `lot.floors[0].spots[3].car.engine`. Ask the object. Amazon.

86. How do you design a shopping cart Low Level Design?

Cart, LineItem, price rules, coupon strategy. Flipkart / Amazon.

87. Interpreter – do you need it?

Almost never in 90 minutes. Say so. Honest Microsoft.

88. How do you design a thread-safe bounded queue?

Mutex + notFull/notEmpty conditions. Amazon / Microsoft CS+LLD.

89. Prototype pattern vs clone.

Clone a configured object instead of re-setup. Rare. Adobe.

90. How do you design an elevator assigner?

Interface `pick(elevators, request)` . Swap algorithms. Amazon.

91. What UML would you draw in 5 minutes?

Classes, key methods, 1–2 relations. No 40 boxes. All LLD.

92. How do you design a rate limit that is fair among users?

Per-user buckets, not one global. Uber / Amazon.

93. Mediator for chat room Low Level Design.

Room keeps users; users do not point to each other. Meta-ish.

94. How do you persist a vending machine state?

Do not in 90 min unless asked; mention snapshot. Amazon.

95. What is composition over inheritance in one example?

Car has Engine; not Car extends Engine. All.

96. How do you design a plugin system?

Interface + register; load implementations. Atlassian / VS Code-lite.

97. Bridge pattern – skip or not?

Skip unless they name it. Use composition. Honest.

98. How do you time-box BookMyShow?

Seats + hold + book in 60 min; payments stub. Flipkart.

99. What questions do you ask before coding LLD?

Users, scale (single process?), must-have use cases, time. All.

100. How do you narrate while coding LLD?

Name the class, why it exists, the next test. Silence is a no-hire. Flipkart / Amazon.